COMPLETE SYSTEM MANUAL & ARCHITECTURE REFERENCE

Enterprise Multi-User Sales Analytics Platform

Comprehensive End-to-End Reference: Dedicated Standalone Admin Console, Multi-Tenant Database Design, User Data Isolation, Role-Based Access Control, JWT Security, Full REST API Documentation, and Production Deployment.

ARCHITECTURE	ADMIN CONSOLE	AUTHENTICATION	TEST VALIDATION
Multi-Tenant Relational	Dedicated Standalone	bcrypt + Signed JWT	16 / 16 Tests Passed

1 Dedicated Standalone Admin Panel & Governance Console

The platform features a **completely separate, standalone executive Admin Panel** designed specifically for platform governance, tenant surveillance, and account moderation. It is visually, functionally, and architecturally separated from the standard user interface.

Visual & Functional Separation Highlights

- **Distinct Executive Dark Theme:** Renders a high-contrast slate background (`#090d16`) with dark card surfaces (`#111827`) and purple/crimson admin badges, immediately establishing an executive administrative posture.
- **Dedicated Admin Authentication Flow:** Direct "Admin" tab on the Login Screen with restricted access enforcement (non-admin attempts are rejected with 403 Access Denied) and 1-click credential auto-fill.
- **Standalone Admin Sub-Navigation:**
  1. **System Dashboard:** Platform aggregates (Total Registered Users, Active Tenants, Suspended Accounts, Platform GMV, and Recent Registrations).
  2. **User Directory:** Live searchable directory with role/status filters, 1-click Activate/Deactivate account toggles, and user inspection actions.
  3. **Sales & User Audit Ledger:** Audit transactions, quantities, customer details, and computed financials for any individual user.
  4. **Database & API Health:** Real-time latency tracking, database engine details (Neon PostgreSQL / MySQL), and connection pool diagnostic health.
- **Instant View Switching:** The top bar includes a "User View" button allowing administrators to inspect standard user views, with a header button ("Admin Console") to return anytime.

2 How Administrators Access the Separate Admin Panel

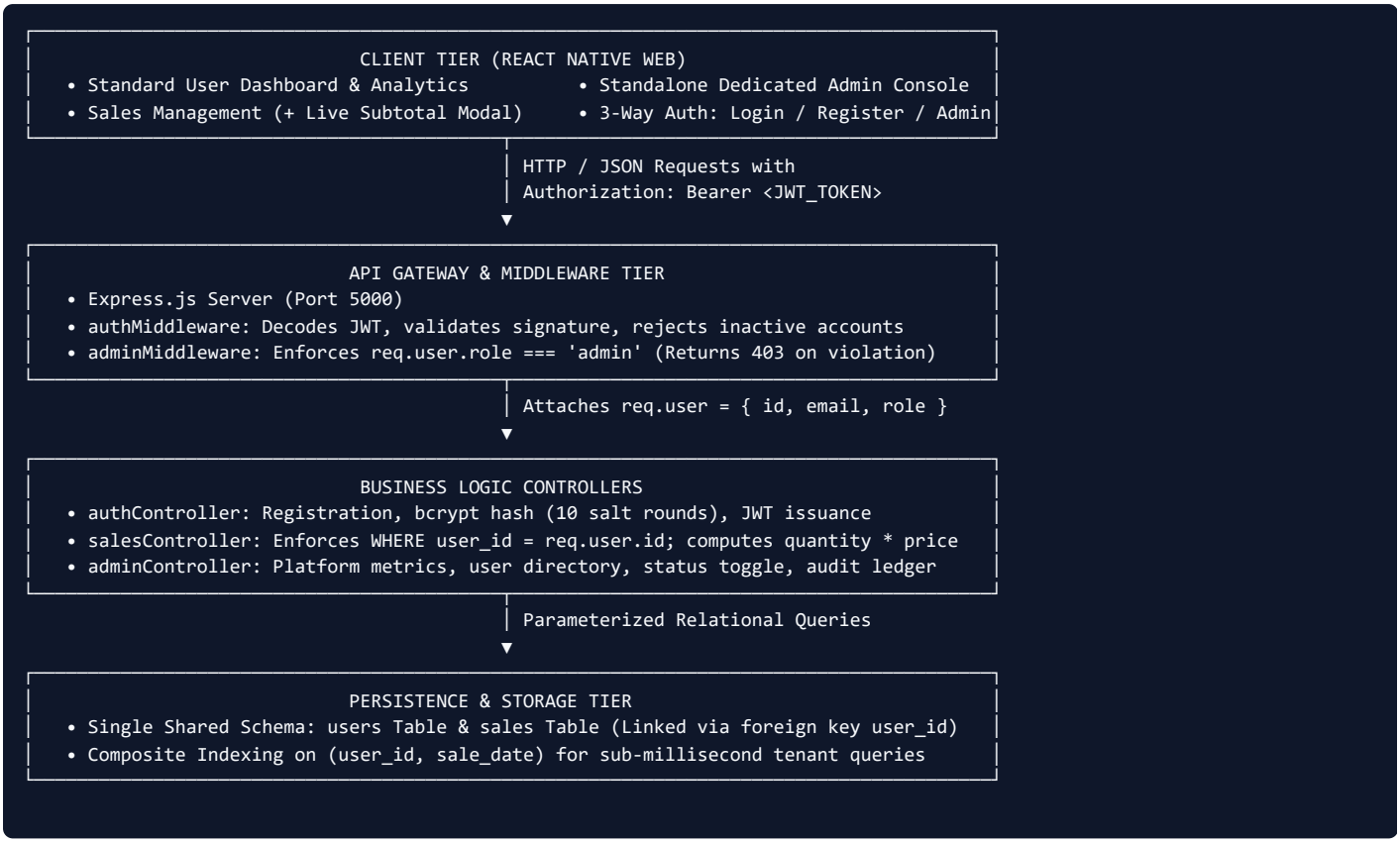
Method A: Direct Admin Portal Login

1. Open the application login screen.
2. Click on the **Admin** tab (shield icon).
3. Click "Fill Credentials" or enter `admin@enterprise.com` / `Admin@123` .
4. Click "Sign In to Admin Console" — launches directly into the Dedicated Admin Panel!

Method B: Header Switcher in Dashboard

1. While logged in as an administrator, look at the top-right header.
2. Click the prominent **[Admin Console]** shield button.
3. The full-screen Dedicated Admin Console immediately activates!
4. Click "User View" anytime to return to standard dashboard testing.

3 Full-Stack Architecture & User Data Isolation Engine



Database Entities: users & sales

TABLE	PRIMARY COLUMNS	FOREIGN KEYS & INDEXES	ISOLATION RULE
`users`	`id, name, email, password_hash, role, status, created_at`	UNIQUE index on `email`	Normal users can only see their own profile; admins view directory without passwords.
`sales`	`id, user_id, customer_name, product_name, category, quantity, price, total_amount, sale_date, region, status`	FK: `user_id -> users(id)` Composite Index: `(user_id, sale_date)`	Enforced strictly via `WHERE user_id = req.user.id` . Server computes `total_amount = quantity * price` .

4 Complete REST API Directory with Payloads

METHOD	ENDPOINT	ACCESS LEVEL	PURPOSE & QUERY FILTER
POST	`/api/auth/register`	Public	Register new user; assigns `role='user'` and `status='active'`.
POST	`/api/auth/login`	Public	Authenticate credentials via bcrypt; returns signed JWT.
GET	`/api/sales`	User / Admin	Fetch sales isolated to authenticated user (`WHERE user_id = req.user.id`).
POST	`/api/sales`	User / Admin	Create sale. Backend computes `total_amount = quantity * price` .
DELETE	`/api/sales/:id`	User / Admin	Delete sale. Blocks IDOR attempts across users with 404.
GET	`/api/dashboard/stats`	User / Admin	Returns user-specific KPI stats (revenue, orders, average value).
GET	`/api/admin/dashboard`	Admin Only	Platform aggregate metrics: total registered users, active, inactive, GMV.
GET	`/api/admin/users`	Admin Only	List all registered users with role, status, sales count, and total revenue.
PUT	`/api/admin/users/:id/status`	Admin Only	Toggle user status (`active` / `inactive`). Deactivated users receive 403.
GET	`/api/admin/users/:id/sales`	Admin Only	Admin audit ledger: view all transactions logged by a target user.

5 Automated Verification & Security Test Suite (16/16 Passed)

#	TEST CASE	TESTED CONDITION	VERIFIED RESULT
1-2	User Registration	POST /api/auth/register	PASS Created with role 'user'
3-4	User Login	POST /api/auth/login	PASS Valid JWT issued
5-7	Sales Creation	POST /api/sales ($3,000)	PASS Server-computed total amount
8	**User Isolation**	GET /api/sales (as User B)	PASS **0 records (User A invisible)**
10	**RBAC Denial Check**	GET /api/admin/users (as User A)	PASS **403 Forbidden returned**
12	Password Privacy	GET /api/admin/users	PASS **Zero passwords/hashes exposed**
14-16	User Deactivation	PUT status / Attempt Login	PASS **403 Deactivated User Blocked**

6 Quick Start, Credentials & Execution Commands

Default Administrator Credentials

Email: admin@enterprise.com
Password: Admin@123
Role: admin | **Status:** active

Default User Account (Example)

Email: usera_1790248291349@analytics.test
Password: Password@123
Role: user | **Status:** active

OPERATION	COMMAND	TARGET PATH	DESCRIPTION
Run Security Tests	`npm run test:api`	Project Root	Executes the 16-point security, RBAC, and data isolation suite.
Build Web Client	`npm run build`	Project Root	Compiles React Native Web bundle into `dist/`.
Run Unified Server	`npm start`	Project Root	Starts the production server on `http://localhost:5000`.
Run Dev Backend	`npm run backend`	Project Root	Runs Express backend with hot-reload on port 5000.
Run Dev Frontend	`npm run web`	Project Root	Runs Expo Web dev server on `http://localhost:8081`.